Operators and Conditional Statements

Computer Programming 1 · Lecture 3

New Uzbekistan University

Week 3 · Fall 2026

Where we are

Last week
Algorithms, pseudocode and flowcharts; then variables, the primitive types, bits and storage, `cin` and `cout`.

Today
The operators that build every expression, and the statements that let a program *choose*.

Next week
Quiz 1, then loops.

Formatting output

The manipulators live in `#include <iomanip>`

Manipulator	What it does	Sticky?
`setw(n)`	Pads the *next* item to width *n*.	No — next item only
`setprecision(n)`	*n* significant digits — or *n* decimals once `fixed` is on.	Yes
`fixed`	Plain decimal notation, no exponent.	Yes
`setfill(c)`	Pads with `c` instead of a space.	Yes
`left` / `right`	Alignment inside the `setw` field.	Yes

Sticky means it stays in force until you change it — for the rest of the program.

`setw`: a width for exactly one item

```
cout << setw(8) << "ab" << "cd" << endl;
cout << setfill('.') << setw(8) << 42 << endl;
cout << setfill(' ') << left << setw(8) << 42 << "|" << endl;
```

Output

```
      abcd
......42
42      |
```

Only `"ab"` was padded. `setfill` and `left`, by contrast, stay on until you switch them back.

`setprecision`: significant digits, or decimals

```
double pi = 3.14159265, big = 1234567.0;

cout << pi << endl;
cout << big << endl;
cout << setprecision(3) << pi << endl;
cout << fixed << setprecision(3) << pi << endl;
cout << fixed << setprecision(2) << 19.999 << endl;
```

`setprecision`: significant digits, or decimals

```cpp
double pi = 3.14159265, big = 1234567.0;

cout << pi << endl;
cout << big << endl;
cout << setprecision(3) << pi << endl;
cout << fixed << setprecision(3) << pi << endl;
cout << fixed << setprecision(2) << 19.999 << endl;
```

Output

```
3.14159
1.23457e+06
3.14
3.142
20.00
```

Default precision is 6 *significant* digits

Which is why `1234567.0` came out in scientific notation. With `fixed`, `setprecision` counts digits *after the point* — and it rounds, it does not cut.

Putting it together — a formatted table

```
cout << left  << setw(12) << "Item"
     << right << setw(6)  << "Qty"
     << right << setw(10) << "Price" << endl;

cout << setfill('-') << setw(28) << "" << endl;
cout << setfill(' ') << fixed << setprecision(2);

cout << left  << setw(12) << "Notebook"
     << right << setw(6)  << 3
     << right << setw(10) << 12.5 << endl;
cout << left  << setw(12) << "Pen"
     << right << setw(6)  << 12
     << right << setw(10) << 1.75 << endl;
```

Output

```
Item           Qty      Price
----------------------------
Notebook         3      12.50
Pen             12       1.75
```

Three ideas: one `fixed` setting for the whole table, a `setw` on *every* column, and `setfill` to draw a rule.

Your turn — predict the output

```cpp
double price = 7.456;

cout << setw(6) << 42 << "|" << endl;
cout << setprecision(2) << price << endl;
cout << fixed << setprecision(2)
     << price << endl;
cout << setw(8) << price << "|" << endl;
```

Your turn — predict the output

```
double price = 7.456;

cout << setw(6) << 42 << "|" << endl;
cout << setprecision(2) << price << endl;
cout << fixed << setprecision(2)
     << price << endl;
cout << setw(8) << price << "|" << endl;
```

Output

```
    42|
7.5
7.46
    7.46|
```

`setprecision(2)` alone gives 2 *significant* digits; after `fixed` it gives 2 *decimals*.

Operators

What an operator is

Definition

An operator combines one or more operands into an expression — and every expression produces a **value** and a **type**.

Family	Operators	Result type
Arithmetic	`+ - * / %`	a number
Assignment	`= += -= *= /= %=`	the value stored
Increment	`++ --`	a number
Relational	`< <= > >= == !=`	`bool`
Logical	`&& || !`	`bool`

`3 + 4` is an `int`; `3 < 4` is a `bool`. The type decides what you may do next with it.

Arithmetic: five operators

Op	Meaning	Example	Result
`+`	addition	`7 + 2`	`9`
`-`	subtraction	`7 - 2`	`5`
`*`	multiplication	`7 * 2`	`14`
`/`	division	`7 / 2`	`3` `7 / 2.0` → `3.5`
`%`	remainder	`7 % 2`	`1`

The rule behind the pitfall

`int / int` is **integer division**: the fractional part is thrown away, not rounded. `%` needs two integers — `7.5 % 2` does not compile. Negative operands truncate towards zero: `-7 / 2` is `-3` and `-7 % 2` is `-1`.

/ and % are a team

```
int total = 3725;            // seconds

int h = total / 3600;        // 1
int m = (total % 3600) / 60; // 2
int s = total % 60;          // 5

cout << h << " h " << m
     << " min " << s << " s";
```

Output

```
1 h 2 min 5 s
```

Splitting a number into units is / for “how many whole” and % for “what is left”.

Mixing types: one `double` is enough

```
int sum = 7, n = 2;

double a = sum / n;            // 3
double b = sum / 2.0;          // 3.5
double c = static_cast<double>(sum) / n;
double d = static_cast<double>(sum / n);
```

Values

```
a = 3
b = 3.5
c = 3.5
d = 3
```

Convert *before* you divide, not after

In `d` the division already happened in `int`; casting the answer afterwards cannot bring the 0.5 back. The cast has to touch an *operand*.

Precedence: which operator goes first

	Operators	Associativity
highest	`()`	—
	`++ --` (prefix) `! ` unary `- static_cast`	right to left
	`* / %`	left to right
	`+ -`	left to right
	`< <= > >=`	left to right
	`== !=`	left to right
	`&&`	left to right
	`||`	left to right
lowest	`= += -= *= /= %=`	right to left

Two things worth noticing: `&&` beats `||`.

Evaluate it like the compiler does

```
cout << 9 / 2 * 2.0 + 9 % 4;
```

Step	Why	Expression becomes
1	`/` and `*` and `%` first, left to right	`9 / 2` → `4` (integer division)
2	still left to right	`4 * 2.0` → `8.0` (now `double`)
3	`%` binds tighter than `+`	`9 % 4` → `1`
4	finally the `+`	`8.0 + 1` → `9.0`

Output

```
9
```

`9.0` prints as `9`: the default precision shows no trailing zeros.

Assignment, and its compound forms

```
int x = 10;

x += 5;     // x = x + 5  -> 15
x -= 3;     // 12
x *= 2;     // 24
x /= 5;     // 4   (integer division!)
x %= 3;     // 1
```

Read it right to left

`=` is not equality. It computes the right-hand side, then stores it in the name on the left.

`x /= 5` obeys exactly the same integer-division rule — the compound form hides nothing.

++ and --: prefix or postfix

```cpp
int a = 5;
int b = a++;   // use, then increase
cout << a << " " << b << endl;

int c = 5;
int d = ++c;   // increase, then use
cout << c << " " << d << endl;
```

++ and --: prefix or postfix

```
int a = 5;
int b = a++;   // use, then increase
cout << a << " " << b << endl;

int c = 5;
int d = ++c;   // increase, then use
cout << c << " " << d << endl;
```

Output

```
6 5
6 6
```

Both *variables* end at 6. The difference is only the value the expression handed over.

Where it bites

`a++` inside a larger expression changes `a` as a *side effect*. In a condition, that side effect may or may not happen — which is the next slide.

Relational operators produce a `bool`

```cpp
int age = 20;

cout << (age > 18) << endl;    // 1
cout << (age == 25) << endl;   // 0
cout << boolalpha
     << (age != 25) << endl;   // true
```

`<`	less than
`<=`	at most
`>`	greater than
`>=`	at least
`==`	equal
`!=`	not equal

Two things to remember

`cout` prints a `bool` as `1` or `0` until you ask for `boolalpha`. And the comparison needs parentheses inside a `cout`, because `<<` binds tighter than `<`.

Trap — the chain that lies

```
int x = 10;

if (3 < x < 5)        // compiles!
    cout << "in range";
```

Output

```
in range
```

Why

Left to right: `3 < 10` gives `true`, which converts to `1`; then `1 < 5` is `true`. The test never looked at the range at all.

Mathematics allows $3 < x < 5$. C++ needs `if (3 < x && x < 5)`.

Logical operators

a	b	a && b	a || b
true	true	true	true
true	false	false	true
false	true	false	true
false	false	false	false

`!a` flips `a`.

In words

`&&` — **both** must hold.

`||` — **at least one** must hold.

`!` — **not**.

Any number is a condition

`0` is `false`; every other value is `true`. So `if (n % 2)` means "if `n` is odd".

Short-circuit evaluation

```
int x = 4;
if (x > 5 || ++x > 0)
    cout << "yes ";
cout << x << endl;

int y = 4;
if (y > 5 && ++y > 0)
    cout << "yes ";
cout << y << endl;
```

Short-circuit evaluation

```
int x = 4;
if (x > 5 || ++x > 0)
    cout << "yes ";
cout << x << endl;

int y = 4;
if (y > 5 && ++y > 0)
    cout << "yes ";
cout << y << endl;
```

Output

```
yes 5
4
```

In the first, `x > 5` is false, so `++x` *must* be evaluated — and `x` changes.
In the second, `y > 5` is false, so `&&` already knows the answer and `++y` never runs.

The rule

`&&` stops at the first `false`; `||` stops at the first `true`. Anything with a side effect on the right may silently not happen.

Short-circuit is also a safety belt

```
if (c != 0 && a / c > 1)      // safe: division never runs when c == 0
    cout << "big ratio";
```

Swap the two tests and the program can crash

`if (a / c > 1 && c != 0)` divides first. With `c == 0` that is division by zero — undefined behaviour, usually a crash.

Put the *guard* on the left. This is the one place where the order of a condition is not style, but correctness.

Your turn — three expressions

```
int a = 11, b = 4;

cout << a / b << " " << a % b << endl;
cout << a / b * 1.0 << endl;
cout << (a > b && b % 2 == 0) << endl;
```

Your turn — three expressions

```
int a = 11, b = 4;

cout << a / b << " " << a % b << endl;
cout << a / b * 1.0 << endl;
cout << (a > b && b % 2 == 0) << endl;
```

Output

```
2 3
2
1
```

Line 2 is the usual trap: `a / b` is already `2` before `* 1.0` ever happens.

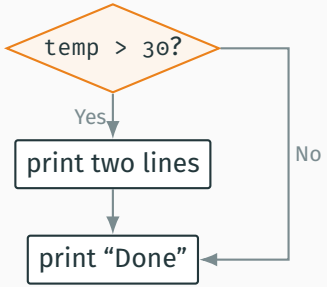
To get `2.75` you must write `a / (b * 1.0)` or `static_cast<double>(a) / b`.

Conditional statements

`if`: run a block only when a condition holds

```
int temperature = 35;

if (temperature > 30) {
    cout << "Hot day" << endl;
    cout << "Drink water" << endl;
}
cout << "Done" << endl;
```



Three pieces of punctuation

Parentheses around the condition. No semicolon after them. Braces around the block — indentation is for humans, braces are for the compiler.

Two `if` mistakes that still compile

A stray semicolon

```
if (score > 90);
    cout << "Excellent";
```

The `if` now controls an *empty* statement. The message prints for everybody.

Assignment instead of comparison

```
if (x = 0)
    cout << "zero";
```

`x` is *set* to 0, and the condition becomes that 0 — false. Your variable is destroyed too.

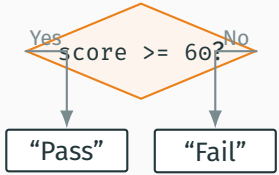
Both are logic errors, not syntax errors

The compiler may warn, but it will not stop. Turn warnings on: `g++ -Wall -Wextra`.

`if-else`: exactly one of two paths

```
int score;
cin >> score;

if (score >= 60) {
    cout << "Pass" << endl;
} else {
    cout << "Fail" << endl;
}
```



Every input takes exactly one branch — so the two branches together must cover *every* case.

`else if` chains: the first true wins

```
int age;
cin >> age;

if (age < 7) {
    cout << "Free";
} else if (age < 18) {
    cout << "Student ticket";
} else if (age >= 60) {
    cout << "Senior ticket";
} else {
    cout << "Full ticket";
}
```

Reading it

Conditions are tested top to bottom. The first one that is true runs — the rest are skipped, even if they are also true.

Why the later tests are short

Reaching `age < 18` already guarantees `age >= 7`. You never need to repeat what the previous test ruled out.

Order is part of the logic

Correct — strictest first

```
if (avg >= 90)
    cout << "Excellent";
else if (avg >= 75)
    cout << "Good";
else
    cout << "Needs work";
```

Wrong — loosest first

```
if (avg >= 75)
    cout << "Good";
else if (avg >= 90)
    cout << "Excellent";
else
    cout << "Needs work";
```

On the right, `"Excellent"` can never print

`avg = 95` already satisfies `avg >= 90` — sorry, `avg >= 75` — and the chain stops there. A branch no input can reach is unreachable code, and the compiler will not report it.

Nested `if`: a decision inside a decision

```
if (hasTicket) {
    if (age >= 18) {
        cout << "Enter";
    } else {
        cout << "Guardian needed";
    }
} else {
    cout << "Buy a ticket";
}
```

Nesting vs &&

Two nested `if`s with no inner `else` say the same thing as one `&&`.

Nest when the branches differ; combine with `&&` when they do not.

Keep nesting shallow — three levels deep is usually a sign that a condition should be given a name.

The dangling `else`

```
int a = 7, b = 3, c = 0;

if (a > b)
    if (c != 0 && a / c > 1)
        cout << "X";
else
    cout << "Y";
```

The dangling `else`

```
int a = 7, b = 3, c = 0;

if (a > b)
    if (c != 0 && a / c > 1)
        cout << "X";
else
    cout << "Y";
```

Output

```
Y
```

The indentation suggests the `else` belongs to the *outer* `if`. It does not.

The rule

An `else` binds to the nearest unmatched `if` — here, the inner one. `a > b` is true, the inner condition is false, so the inner `else` runs and prints `Y`.

Always use braces. Then the question cannot arise.

Name your conditions with `bool`

```
bool heavy  = mass >= 100;
bool bulky  = l >= 1000 || w >= 1000
           || volume >= 1000000;

if (bulky && heavy) {
    cout << "Both";
} else if (bulky) {
    cout << "Bulky";
} else if (heavy) {
    cout << "Heavy";
} else {
    cout << "Neither";
}
```

Why this is better

The chain now reads like the specification. Each `bool` is tested once and reused.

Watch the type

`int volume = l * w * h;` overflows for large sides. Compute it as

```
long long volume =
 static_cast<long long>(l) *
w * h;
```

Worked example — classify a number

```
int n;
cin >> n;

if (n % 15 == 0) {
    cout << "both";
} else if (n % 3 == 0) {
    cout << "three";
} else if (n % 5 == 0) {
    cout << "five";
} else {
    cout << "neither";
}
```

Input	Output
30	both
9	three
20	five
7	neither

Test the most specific case first — move `n % 3` to the top and `30` prints `three`.

The conditional mistakes that cost marks

What you write	What happens
`if (x = 5)`	Assigns 5, tests 5 — always true, and `x` is lost.
`if (x > 0);`	Empty body; the next line always runs.
`if (3 < x < 5)`	Always true. Use `3 < x && x < 5`.
`if (a / c > 1 && c != 0)`	Divides before checking — guard first.
Loosest test first	Later branches become unreachable.
No braces + nested `if`	The `else` binds to the inner `if`.
`if (x == 0.1 + 0.2)`	Never true — compare doubles with a tolerance.

Every one of these compiles. The compiler checks your grammar, never your meaning.

Quiz 1 revision

Quiz 1 — what to expect

Format	
Section A — multiple choice	30
Section B — fill in the gaps	40
Section C — trace and write	30
Total	**100**

Week 4 · 35 minutes · no notes, phones or computers.

Revision 1 — quick fire (a)

1. The hexadecimal literal `0x1B` has the decimal value: A. 11 B. 27 C. 31 D. 127

2. What does this print?

```
cout << 9 / 2 * 2.0 + 9 % 4;
```

A. 9 B. 10.5 C. 9.5 D. 8

Revision 1 — quick fire (a)

1. The hexadecimal literal `0x1B` has the decimal value: A. `11` B. `27` C. `31` D. `127`

2. What does this print?

```
cout << 9 / 2 * 2.0 + 9 % 4;
```

A. `9` B. `10.5` C. `9.5` D. `8`

Answers

1. B — $1 \times 16 + 11 = 27$. **2. A** — `9/2` is `4`, `4*2.0` is `8.0`, `9%4` is `1`, total `9.0`, printed as `9`.

Revision 1 — quick fire (b)

3. What does this print?

```
int x = 4;
if (x > 5 || ++x > 0) { cout << "ok "; }
cout << x;
```

A. ok 4 B. ok 5 C. 5 D. 4

4. Which expression does **not** evaluate to `3.5`? A. `7 / 2.0` B. `static_cast<double>(7) / 2`
C. `static_cast<double>(7 / 2)` D. `7.0 / 2`

Revision 1 — quick fire (b)

3. What does this print?

```
int x = 4;
if (x > 5 || ++x > 0) { cout << "ok "; }
cout << x;
```

A. ok 4 B. ok 5 C. 5 D. 4

4. Which expression does **not** evaluate to `3.5`? A. `7 / 2.0` B. `static_cast<double>(7) / 2` C. `static_cast<double>(7 / 2)` D. `7.0 / 2`

Answers

3. B — the left test is false, so `||` must evaluate `++x`: `x` becomes 5 and the condition is true. **4. C** — the division happens in `int` first, giving 3; the cast then turns 3 into `3.0`.

Revision 1 — quick fire (c)

5. What does this print? *Hint: in the ASCII table `'A'` = 65.*

```
char c = 'D' - 1;
cout << c << (c + 1);
```

A. CD B. C68 C. 6768 D. C D

Revision 1 — quick fire (c)

5. What does this print? *Hint: in the ASCII table `'A'` = 65.*

```
char c = 'D' - 1;
cout << c << (c + 1);
```

A. `CD` B. `C68` C. `6768` D. `C D`

Answer: B

`'D'` is 68, so `c` holds 67 — and because `c` is a `char`, `cout` prints the letter `C`.

`c + 1` is *arithmetic*: the `char` is promoted to `int`, so the expression is the `int` 68 and prints as digits.

To print the next letter you need `static_cast<char>(c + 1)`.

Revision 2 — fill in the gaps

Centimetres to metres. Input `345` must print `3 m 45 cm`.

```
#include <___(1)___>
using namespace std;
int main() {
    int total;
    cin ___(2)___ total;
    int m  = total ___(3)___ 100;
    int cm = total ___(4)___ 100;
    cout << m << " m " << cm << " cm" << endl;
    return 0;
}
```

Revision 2 — fill in the gaps

Centimetres to metres. Input `345` must print `3 m 45 cm`.

```
#include <___(1)___>
using namespace std;
int main() {
    int total;
    cin ___(2)___ total;
    int m  = total ___(3)___ 100;
    int cm = total ___(4)___ 100;
    cout << m << " m " << cm << " cm" << endl;
    return 0;
}
```

Answers

(1) `iostream` (2) `>>` (3) `/` (4) `%`

Revision 3 — fill in the gaps, with a decision

Four scores. Input `85 90 78 80` must print `Average: 83.25, rounded: 83, band: B`. Band `A` for 85 or more, `B` for 70 or more, otherwise `C`; an average outside 0–100 is invalid.

```
int a, b, c, d;   cin >> a >> b >> c >> d;
double avg = ___(5)___(a + b + c + d) / 4;   // not integer division
char band;
if (___(6)___) {                  // outside 0..100
    cout << "Invalid" << endl;
} else {
    if (avg ___(7)___ 85)  band = 'A';
    else if (avg >= 70)  band = 'B';
    else                 band = 'C';
    cout << "Average: " << avg << ", rounded: "
         << ___(8)___(avg + 0.5) << ", band: " << band << endl;
}
```

Revision 3 — fill in the gaps, with a decision

Four scores. Input `85 90 78 80` must print `Average: 83.25, rounded: 83, band: B`. Band `A` for 85 or more, `B` for 70 or more, otherwise `C`; an average outside 0–100 is invalid.

```
int a, b, c, d;   cin >> a >> b >> c >> d;
double avg = ___(5)___(a + b + c + d) / 4;   // not integer division
char band;
if (___(6)___) {                  // outside 0..100
    cout << "Invalid" << endl;
} else {
    if (avg ___(7)___ 85)  band = 'A';
    else if (avg >= 70)  band = 'B';
    else                 band = 'C';
    cout << "Average: " << avg << ", rounded: "
         << ___(8)___(avg + 0.5) << ", band: " << band << endl;
}
```

Answers: (5) `static_cast<double>` (6) `avg < 0 || avg > 100` (7) `>=` (8) `static_cast<int>`

Revision 4 — trace exactly

```
int a = 9, b = 4, c = 0;
double d = a / b + a % b / 2.0;
if (a > b)
    if (c != 0 && a / c > 1)
        cout << "P";
    else
        cout << "Q";
cout << " " << d;
char ch = 'c' - 2;
if (ch >= 'a' && ch <= 'z')
    ch = ch - 32;
cout << " " << ch << " " << (ch < 'B') << endl
     ;
```

Revision 4 — trace exactly

```
int a = 9, b = 4, c = 0;
double d = a / b + a % b / 2.0;
if (a > b)
    if (c != 0 && a / c > 1)
        cout << "P";
    else
        cout << "Q";
cout << " " << d;
char ch = 'c' - 2;
if (ch >= 'a' && ch <= 'z')
    ch = ch - 32;
cout << " " << ch << " " << (ch < 'B') << endl
     ;
```

Output

```
Q 2.5 A 1
```

- `a / b` is 2; `a % b` is 1; `1 / 2.0` is `0.5` → `d = 2.5`
- `c != 0` is false, so the division is skipped and the `else` prints `Q`
- `'c' - 2` is `'a'`; it is lowercase, so −32 makes it `'A'`
- `'A'` (65) < `'B'` (66) → `1`

Revision 5 — write the program

Lap time with a split. Read a time as minutes and seconds (`mm ss`) and an extra split `s` in seconds. Print the total as minutes and seconds. Minutes simply keep growing — do **not** wrap at 60. Use only `int`, `/` and `%`.

```
5 20 400   →   12 min 0 s
3 50 95    →   5 min 25 s
0 45 200   →   4 min 5 s
```

5 minutes — write it on paper, exactly as you would in the quiz.

Revision 5 — one correct answer

```cpp
#include <iostream>
using namespace std;
int main() {
    int mm, ss, s;
    cin >> mm >> ss >> s;

    int total = mm * 60 + ss + s;

    int minutes = total / 60;
    int seconds = total % 60;

    cout << minutes << " min "
         << seconds << " s" << endl;
    return 0;
}
```

The method

Convert everything to the smallest unit first, add, then split back with `/` and `%`.

No wrapping

We never take `minutes % 60`, so 12 minutes stays 12.

How to prepare this week

1. Practice C++ on paper — the quiz has no compiler.
2. For each code fragment, write the output *before* you run it, then check.
3. Practise the numerical conversions in all directions, e.g., decimal $\leftrightarrow$ binary, decimal $\leftrightarrow$ hexadecimal etc.
4. Learn the precedence order for conditionals, primitive data type conversions, and `char` arithmetic operations.
5. Write three short programs from scratch: unit splitting, a grade band, and a validity check.

Wrap-up

What to take away

Operators

- Precedence: `* / %` before `+ -`, `&&` before `||`, `<<` before every comparison.
- `&&` stops at the first false, `||` at the first true — side effects may vanish.

Conditionals

- The first true branch of a chain wins — so order the tests from strictest to loosest.
- An `else` binds to the nearest unmatched `if`; braces settle it.
- Name complicated conditions with `bool` variables.

And `fixed`, `setprecision`, `setw` make the output look like a result rather than a dump.

Questions?